

Informe:

Tema: Diseño Funcional en Common Lisp y Estudio Comparativo con Haskell.

Grupo: 49

Integrantes:

1. Miño Martin
2. Geronimo Agustín Tabares
3. Zeniquel Santiago de Jesús
4. Miño Maidana Ulises
5. Soto Gastón Dante

FUNDAMENTOS DE DISEÑO

1. **Requerimiento (Estados de Transición):** La función evolucionó a través de varias iteraciones para asegurar su robustez. Inicialmente, el diseño solo contemplaba el estado 'en-rojo', lo que provocaba que la función devolviera NIL si la condición no se cumplía. Posteriormente se intentaron esquemas repetitivos con múltiples condicionales explícitos. La versión final, sin embargo, elimina el código repetido utilizando las funciones `member` y `concatenate`, logrando un diseño mucho más mantenible y escalable. Además, se incluyeron validaciones de tipo que aseguran que se devuelva una 'accion-por-defecto si los colores ingresados son inválidos, cumpliendo con todo lo solicitado de forma pura.
2. **Requerimiento (Temporizador Automático):** El desarrollo de esta función requirió refactorizaciones clave para representar el ciclo semafórico. Las primeras versiones fallaban al distinguir solo dos estados (rojo y verde), ignorando el amarillo y el ciclo completo. Para corregirlo, se incorporó aritmética modular sobre un ciclo de 216 segundos. Durante el proceso, se intentaron implementar variables temporales utilizando variables globales (`setq`) y locales (`let`). Sin embargo, para respetar el paradigma funcional puro, la versión final dejó de lado la función `let`, integrando la validación del código y aplicando el cálculo modular directamente en las guardas de la estructura condicional.
3. **Requerimiento (Registro de Cambios):**

El diseño de la función de registro de cambios requirió integrar herramientas nativas con librerías externas, priorizando la pureza y la eficiencia en el manejo de datos:

- **Conversión de Epoch a Timestamp:** Se determinó que la utilización de (`local-time:unix-to-timestamp epoch`) es la metodología más limpia para transformar un formato de tiempo universal estandarizado en un objeto de tiempo fácilmente manipulable.
- **Manejo de Símbolos (`symbol-name`):** Al recibir los parámetros `color-anterior` y `color-nuevo` en forma de símbolos (por ejemplo, 'rojo' o 'verde'), el código se mantiene idiomático dentro de las convenciones de Lisp, ya que los símbolos son computacionalmente más eficientes y limpios que las cadenas de texto para representar estados finitos. La instrucción (`symbol-name color-anterior`) resulta crucial aquí, ya que extrae la representación en texto del símbolo de forma segura, evitando arrastrar caracteres especiales o prefijos de paquetes y garantizando un registro final perfectamente estructurado.

- **Uso de concatenate 'string:** La decisión de emplear esta función responde a un requerimiento de diseño directo: lograr unir piezas estáticas de texto con variables dinámicas de forma consecutiva e inmutable, sin generar variables intermedias.

Explicación Teórica y Diseño: Requerimiento 4 (Métricas del Ciclo Semafórico)

Este requerimiento se dividió en dos funciones complementarias, ambas diseñadas con estricta adherencia a las reglas del paradigma declarativo:

1. Función duracion-ciclo

- **Análisis y Estrategia:** Se implementó como una función de naturaleza estrictamente **pura**, garantizando que su valor de retorno dependa única y exclusivamente de los argumentos de entrada (duración del rojo, amarillo y verde) sin alterar el entorno. La estrategia de control utilizada es **Directa/Simple**, ya que el cálculo del ciclo total se resuelve mediante una única operación aritmética abstracta (la suma de los tres estados). Adicionalmente, se blindó la función con predicados de tipo (numberp) para atajar entradas inválidas preservando la pureza.

2. Función recomendacion

- **Análisis y Estrategia:** Al igual que su predecesora, se estructuró como una función **pura**, dado que el diagnóstico emitido está condicionado exclusivamente por el valor de duración calculado previamente. La estrategia de control aplicada es de **Predicado con toma de decisión**. Utiliza la estructura cond para evaluar lógicamente el valor recibido contrastándolo contra los límites operacionales establecidos por los estándares de ingeniería de tráfico (ciclos menores a 35 segundos o mayores a 150 segundos).

Explicación Teórica y Diseño: Requerimiento 5 (Proyección de Ciclos Temporales)

El diseño final de esta función prioriza la escalabilidad, la reutilización de código y el cumplimiento estricto del paradigma funcional puro, evitando variables intermedias mediante la composición de funciones.

Las decisiones arquitectónicas clave fueron:

- **Parametrización Dinámica:** La función recibe los tiempos específicos del semáforo (rojo, amarillo y verde) como parámetros, en lugar de utilizar valores fijos o "hardcodeados" en el cuerpo del código. Esto garantiza que el módulo sea completamente reutilizable y escalable ante futuros

cambios en la configuración del semáforo, sin necesidad de modificar su lógica interna.

Homogeneización y Composición Funcional: Se realiza una conversión inicial de los minutos ingresados a segundos ($\text{minutos} \times 60$) para unificar la unidad de medida con el resto del sistema. Seguidamente, se aplica el principio de composición delegando el cálculo del denominador a la función `duracion-ciclo` (desarrollada en el Requerimiento 4). Esto mejora significativamente la mantenibilidad del código.

Estrategia de Control (Orden Superior): Para proyectar la cantidad de ciclos, se divide el tiempo transcurrido por la duración de un ciclo. Dado que la regla de negocio exige contar exclusivamente "ciclos completos", se descarta cualquier aproximación decimal aplicando la función de orden superior `truncate`.

Blindaje de Datos (Robustez): Se incorporaron guardias de seguridad utilizando predicados de tipo y signo (`numberp`, `integerp`, `> 0`). Esto previene excepciones a nivel de sistema si el programa recibe cadenas de texto o tiempos negativos, garantizando una ejecución segura.

[Explicación Teórica y Diseño: Requerimiento 6 \(Informe de Distribución Temporal\)](#)

El módulo de distribución temporal se diseñó respetando estrictamente el paradigma declarativo. Su propósito es calcular el porcentaje que representa cada color dentro del ciclo total del semáforo.

- **Naturaleza y Estrategia:** Se implementó como una función **pura** (su salida depende exclusivamente de los argumentos ingresados) y su impacto es **no destructivo** (no modifica el estado global del sistema). La estrategia utilizada es la **composición funcional directa**, delegando la sumatoria total a la función `duracion-ciclo` (desarrollada en el Requerimiento 4).
- **Manejo de Tipos (Casteo):** Se determinó el uso explícito de la función `float` para asegurar que los porcentajes matemáticos se representen de forma precisa y legible en punto flotante.

[Explicación Teórica y Diseño: Requerimiento 7 \(Aseguramiento de la Calidad\)](#)

El Requerimiento 7 se concibió como un entorno de pruebas automatizado (QA) para validar la solidez del sistema ante escenarios de uso normal y casos de error extremo.

- **Naturaleza Impura:** A diferencia del resto de los módulos desarrollados en este trabajo práctico, esta función es intencionalmente **impura**. Su objetivo no es devolver un valor matemático encapsulado, sino generar "efectos colaterales" visuales imprimiendo los resultados de los tests directamente en la consola del usuario mediante el uso de las instrucciones `format` y `print`.

BITACORA DE BUGS

Evolución y Depuración: Requerimiento 1 (Estados de Transición)

Durante el desarrollo de esta función, el equipo atravesó cuatro iteraciones lógicas hasta alcanzar la pureza y eficiencia requeridas:

Iteración 1: Aproximación inicial (Con errores lógicos)

- **Análisis:** Esta versión presentaba un fallo crítico de completitud. Solo contemplaba el estado 'en-rojo'. Al utilizar un `if` sin una rama falsa explícita, cualquier otro color ingresado provocaba que la función devolviera silenciosamente `NIL`, rompiendo el flujo del programa en un escenario real.

```
> (defun transicion (color-actual cambiar-a)
  (if (equal color-actual 'en-rojo)
      (list color-actual "cambiar-a-verde")))

TRANSICION
> (transicion 'en-verde 'amarillo)
NIL
>
```

Iteración 2: Incorporación de validación de destino (Incompleta)

- **Análisis:** Se intentó solucionar el problema anterior agregando una validación estricta del color destino mediante el operador lógico `and`. Sin embargo, la estructura seguía siendo deficiente: se forzó explícitamente la devolución de `NIL` para el resto de los casos válidos, ignorando por completo el resto del ciclo semafórico.

```
> (defun transicion (color-actual cambiar-a)
  (cond
    ((and (equal color-actual 'en-rojo) (equal cambiar-a 'verde))
      (list color-actual "cambiar-a-verde"))
    (T NIL)))

TRANSICION
> (transicion 'en-amarillo 'rojo)
NIL
>
```

Iteración 3: Aproximación funcional de fuerza bruta (Poco óptima)

- **Análisis:** Desde el punto de vista funcional, este código ya cumplía con los requisitos básicos. Contemplaba todos los estados de destino y devolvía una acción por defecto ante errores. No obstante, arquitectónicamente

presentaba una alta redundancia (repetición innecesaria de bloques `list`), lo que dificultaba su mantenimiento y escalabilidad a futuro.

```
> (defun transicion (color-actual cambiar-a)
  (cond
    ((equal cambiar-a 'rojo) (list color-actual "cambiar-a-rojo"))
    ((equal cambiar-a 'amarillo) (list color-actual "cambiar-a-amarillo"))
    ((equal cambiar-a 'verde) (list color-actual "cambiar-a-verde"))
    (t (list color-actual 'accion-por-defecto))))
TRANSICION
> (transicion 'azul 'negro)
(AZUL ACCION-POR-DEFECTO)
> |
```

-
- **Iteración 4: Refactorización y versión final**
- **Análisis:** En la versión definitiva se erradicó por completo la repetición de código. Se implementó la función de pertenencia `member` para validar los conjuntos de datos, y se combinó `concatenate` con `symbol-name` para construir las respuestas dinámicamente. Además, se agregaron guardias defensivos estructurales para atajar parámetros inválidos de entrada, logrando un código robusto, escalable y 100% puro.

```
> (defun transicion (color-actual cambiar-a)
  (cond
    ((not (member color-actual '(rojo amarillo verde)))
     "Error: Color actual invalido.")
    ((not (member cambiar-a '(rojo amarillo verde)))
     (list color-actual 'accion-por-defecto))
    (t
     (list color-actual
            (concatenate 'string
                          "cambiar-a-"
                          (symbol-name cambiar-a))))))
TRANSICION
> (transicion 'violeta 'verde)
"Error: Color actual invalido."
```

-

[Evolución y Depuración: Requerimiento 2 \(Temporizador Automático\)](#)

El desarrollo del temporizador automático requirió múltiples refactorizaciones para modelar matemáticamente el ciclo semafórico sin violar el paradigma funcional:

Iteración 1: Condicional Binario Incompleto

- **Análisis:** La primera versión era sumamente básica y deficiente. Utilizaba un `if` simple que solo distinguía dos estados, sin considerar la luz amarilla ni el comportamiento cíclico del semáforo a lo largo del tiempo.

```

> (defun timer (tiempo)
  (if (< tiempo 90)
      'rojo
      'verde))
TIMER
> (timer 92)
VERDE

```

Iteración 2: Aproximación con Módulo y Conflicto de Variables

- **Análisis:** Se introdujo la aritmética modular (`mod tiempo 216`) para calcular el instante dentro del ciclo, y se incluyeron los tres colores. Sin embargo, se cometió un error de diseño al intentar usar una variable local llamada `t` en el `let`, lo cual es peligroso en Lisp ya que `T` es una constante reservada del sistema para representar la verdad (`True`).

```

> (defun timer (tiempo)
  (let ((t (mod tiempo 216)))
    (cond
      ((< t 90) 'rojo)
      ((< t 96) 'amarillo)
      (T 'verde))))
TIMER
> (timer 10)
Error: can't assign/bind to constant - T

```

Iteración 3: Variables Globales y Redundancia Lógica

- **Análisis:** El código funcionaba correctamente en su lógica de tiempos, pero arquitectónicamente era destructivo. Se utilizó `setq` para declarar la variable `x`, lo que generaba un efecto colateral global. Además, las comparaciones presentaban redundancia explícita innecesaria (ej: `(and (>= x 90) (< x 96))`).

```

> (defun timer (tiempo)
  (setq x (mod tiempo 216))
  (cond
    ((and (>= x 0) (< x 90)) 'rojo)
    ((and (>= x 90) (< x 96)) 'amarillo)
    ((and (>= x 96) (< x 216)) 'verde)))
TIMER
> (timer 100)
VERDE
> x
100

```

Iteración 4: Limpieza Parcial (Uso de `let` no permitido)

- **Análisis:** Se mejoró la claridad de las condiciones, eliminando las comparaciones redundantes. También se encapsuló la variable usando un bloque local `let` con un nombre más descriptivo (`instante`). Si bien esto solucionaba el impacto destructivo del `setq`, seguía violando las reglas estrictas de pureza funcional exigidas por la cátedra para esta fase.

```
> (defun timer (tiempo)
  (let ((instante (mod tiempo 216)))
    (cond
      ((< instante 90) 'rojo)
      ((< instante 96) 'amarillo)
      (t 'verde))))

TIMER
> (timer 215)
VERDE
~ |
```

Iteración 5: Refactorización Funcional Pura y Definitiva

- **Análisis:** En la versión final se dejó completamente de lado la función `let` para cumplir estrictamente con el paradigma funcional. La operación modular se compuso directamente dentro de las guardas del `cond`. Además, se blindó el sistema agregando validaciones de tipo (`numberp`) y restricciones matemáticas (`< tiempo 0`) para evitar colapsos ante datos de entrada inválidos.

```
> (defun timer (tiempo)
  (cond
    ((not (numberp tiempo))
     "Error: El tiempo debe ser un valor numerico.")
    ((< tiempo 0)
     "Error: El tiempo no puede ser negativo.")
    ((< (mod tiempo 216) 90)
     'rojo)
    ((< (mod tiempo 216) 96)
     'amarillo)
    (t 'verde)))

TIMER
> (timer 'letra-A)
>Error: El tiempo debe ser un valor numerico."
~ |
```

[Evolución y Depuración: Requerimiento 3 \(Registro de Cambios\)](#)

La implementación de este módulo presentó desafíos orientados principalmente a la configuración del entorno y la gestión de dependencias externas, evolucionando en tres etapas clave hasta alcanzar su diseño definitivo:

Iteración 1: Ejecución sin persistencia de dependencias (Fallo de Entorno)

- **Análisis:** Tras instalar el gestor de paquetes Quicklisp, se intentó probar la lógica de tiempo, pero el intérprete lanzaba una excepción. El problema era de persistencia: la librería no se estaba cargando automáticamente en la sesión actual. Se resolvió agregando explícitamente los comandos de carga (`find-package :ql`) y (`ql:quickload "local-time"`) al inicio del flujo.

```
Break 4 [6]> (ql:quickload "local-time")
(print (local-time:now))

*** - READ en
      #<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM>
      #<IO TWO-WAY-STREAM #<INPUT UNBUFFERED FILE-STREAM CHARACTER @5> #<OUTPUT UNBUFFERED FILE-STREAM>
      : no existe ningún paquete con el nombre "QL"
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort debug loop
ABORT      :R3      Abort debug loop
ABORT      :R4      Abort debug loop
Break 5 [7]> |
```

Iteración 2: Fallo de compilación externa (Error de PATH)

- **Análisis:** Al intentar automatizar el desarrollo, el editor Sublime Text no reconocía las instrucciones para ejecutar el archivo. Esto se debía a que las variables de entorno (PATH) del sistema operativo no encontraban el compilador en la ruta predefinida. La solución fue modificar el PATH y redirigirlo a la ubicación correcta del ejecutable de Lisp.

```
1  [
2  {
3    "keys": ["alt+ctrl+h"],
4    "command": "repl_open",
5    "args": {
6      "type": "subprocess",
7      "encoding": "utf8",
8      "cmd": [ "C:\\clisp-2.49\\clisp.exe",
9        "-i",
10       "C:\\Users\\santi\\quicklisp\\setup.lisp",
11       "-repl",
12       "$file"],
13     "cwd": "$file_path",
14     "external_id": "lisp",
15     "syntax": "Packages/Lisp/Lisp.tmLanguage"
16   }
17 }
18 ]
```

Iteración 3: Diseño Técnico e Implementación Final Pura

- **Análisis:** Solucionados los problemas de entorno, se estructuró la función priorizando la eficiencia. Se empleó (`local-time:unix-to-timestamp epoch`) por ser la forma más limpia de transformar el tiempo universal.

Para mantener el código idiomático en Lisp, los parámetros de color se manejaron como símbolos (ej. 'rojo), ya que son más eficientes que las cadenas de texto. Finalmente, se usó `symbol-name` para extraer su texto de forma segura y se unió todo inmutablemente mediante `concatenate 'string`.

Evolución y Depuración: Requerimiento 5 (Proyección de Ciclos Temporales)

El desarrollo de este requerimiento fue uno de los más iterativos del proyecto, atravesando distintas estrategias de control (recursividad simple, recursividad de cola y funciones de orden superior) hasta alcanzar su diseño definitivo:

Iteración 1: Tipo de dato incorrecto y valores fijos (Hardcoding)

- **Análisis:** La primera aproximación intentó resolver el problema dividiendo los minutos directamente por un valor fijo (3.6). El error conceptual aquí fue que la división devolvía un número de punto flotante (decimal), lo cual violaba la lógica del dominio, ya que un semáforo solo completa ciclos enteros y no puede retornar fracciones de ciclo.

```
(defun ciclos-por-tiempo(minutos)
  (/ minutos 3.6))
```

```
CICLOS-POR-TIEMPO
[2]> (ciclos-por-tiempo 10)

2.777778
[3]> (ciclos-por-tiempo 3.6)

1.0
```

Iteración 2: Desbordamiento de Pila (Stack Overflow) por Recursión Simple

- **Análisis:** Buscando una solución elegante para iterar, se implementó una función recursiva simple utilizando `(+ 1 (llamada-recursiva))`. Aunque matemáticamente funcionaba para valores pequeños, la operación de suma quedaba pendiente en la pila de llamadas del sistema operativo esperando el retorno de la recursión. Al someter la función a pruebas de estrés con tiempos muy grandes, la memoria colapsó provocando un *Program stack overflow*.

```
[13]> (defun ciclos-por-tiempo-v2 (minutos)
  (let ((duracion-ciclo 3.6))
    (if (< minutos duracion-ciclo)
      0
      (+ 1 (ciclos-por-tiempo-v2 (- minutos duracion-ciclo))))))

CICLOS-POR-TIEMPO-V2
[14]> (ciclos-por-tiempo-v2 1000)

277
[15]> (ciclos-por-tiempo-v2 1000000)

*** - Program stack overflow. RESET

[16]>
```

Iteración 3: Recursividad de Cola (Tail Recursion) vs. Orden Superior

- **Análisis:** El problema de memoria se mitigó reestructurando el algoritmo hacia una *Tail Recursion* auxiliada por un contador, permitiendo al compilador optimizar la memoria. Sin embargo, tras un debate arquitectónico, el equipo determinó que este enfoque sacrificaba legibilidad, haciendo el código innecesariamente extenso. Se decidió abandonar la recursividad en favor de las funciones de orden superior (truncate), logrando el mismo resultado en una sola línea y de forma mucho más entendible.

```
[18]> (ciclos-por-tiempo 0)

0
[19]> (ciclos-por-tiempo -3)

0
```

Iteración 4: Diseño Funcional Puro, Composición y Blindaje (Versión Final)

- **Análisis:** En la versión definitiva se erradicaron las variables locales innecesarias y las conversiones redundantes. Se parametrizaron los tiempos del semáforo para garantizar la escalabilidad y se aplicó composición funcional directa llamando a la función `duracion-ciclo` (del Requerimiento 4) para calcular el denominador. Finalmente, se detectó que si el usuario ingresaba un texto el programa colapsaba. Para solucionarlo, se blindó el sistema agregando validaciones estrictas de tipo y signo (`numberp`, `integerp`, `> 0`) evitando excepciones fatales.

```
[2]> (defun ciclos-por-tiempo (minutos duracion-rojo duracion-amarillo duracion-verde)
  (truncate (/ (* minutos 60)
    (duracion-ciclo duracion-rojo duracion-amarillo duracion-verde)))
  )

CICLOS-POR-TIEMPO
[3]> (ciclos-por-tiempo -16 "hola" 8 120)

*** - +: "hola" is not a number
Es posible continuar en los siguientes puntos:
```

Versión final con validación:

```
;
; FUNCION: ciclos-por-tiempo
; NATURALEZA: Pura
; ESTRATEGIA DE CONTROL: Orden Superior (truncate)
; IMPACTO: No destructiva
;
; =====
(defun ciclos-por-tiempo (minutos duracion-rojo duracion-amarillo duracion-verde)
  (if (and (numberp minutos) (not (< minutos 0))
        (integerp duracion-rojo) (> duracion-rojo 0)
        (integerp duracion-amarillo) (> duracion-amarillo 0)
        (integerp duracion-verde) (> duracion-verde 0))
      (truncate (/ (* minutos 60)
                    (duracion-ciclo duracion-rojo duracion-amarillo duracion-verde)))
      "ERROR: parametros invalidos"))
```

Evolución y Depuración: Requerimiento 6

El algoritmo atravesó dos etapas clave, enfocadas en mejorar la seguridad del código y alcanzar la pureza absoluta del paradigma:

Iteración 1: Aproximación inicial con variables locales (Falta de pureza y validación)

- **Análisis:** La primera versión del código funcionaba matemáticamente de manera correcta. Sin embargo, presentaba dos deficiencias arquitectónicas: carecía de barreras protectoras contra datos inválidos y utilizaba un bloque `let` para almacenar la variable `total`. En el contexto del paradigma funcional estricto, crear ese estado local interrumpe la fluidez de la composición pura.

```
> (defun distribucion-porcentual (t-rojo t-amarillo t-verde)
  (let ((total (duracion-ciclo t-rojo t-amarillo t-verde)))
    (list
      (list 'ROJO (float (* (/ t-rojo total) 100)))
      (list 'VERDE (float (* (/ t-verde total) 100)))
      (list 'AMARILLO (float (* (/ t-amarillo total) 100)))))
  DISTRIBUCION-PORCENTUAL)
> (distribucion-porcentual 90 6 120)
((ROJO 41.66666666666667) (VERDE 55.55555555555556) (AMARILLO 2.7777777777777777))
```

Iteración 2: Refactorización Funcional Pura y Blindaje (Versión Final)

- **Análisis:** Para alcanzar la pureza absoluta exigida por los estándares del proyecto, se erradicó el uso del `let`. La función se refactorizó utilizando composición funcional directa, invocando `(duracion-ciclo t-rojo t-amarillo t-verde)` dinámicamente en cada cálculo, logrando un código alineado con la inmutabilidad de Lisp. Adicionalmente, se incorporaron sentencias `cond` para validar que los parámetros ingresados sean estrictamente numéricos y mayores a cero, evitando fallos en la ejecución.

```

> (defun distribucion-porcentual (t-rojo t-amarillo t-verde)
  (cond
    ((not (and (numberp t-rojo) (numberp t-amarillo) (numberp t-verde)))
      "Error: Debe ingresar valores numericos en segundos.")
    ((or (<= t-rojo 0) (<= t-amarillo 0) (<= t-verde 0))
      "Error: Los tiempos ingresados deben ser mayores a cero.")
    (t
      (list
        (list 'ROJO (float (* (/ t-rojo (duracion-ciclo t-rojo t-amarillo t-verde)) 100)))
        (list 'VERDE (float (* (/ t-verde (duracion-ciclo t-rojo t-amarillo t-verde)) 100)))
        (list 'AMARILLO (float (* (/ t-amarillo (duracion-ciclo t-rojo t-amarillo t-verde)) 100)))
      )
    )
  )
DISTRIBUCION-PORCENTUAL
> (distribucion-porcentual -10 6 120)
>Error: Los tiempos ingresados deben ser mayores a cero."
>

```

Evolución y Depuración: Requerimiento 7

Durante el desarrollo de las rutinas de prueba, se detectó una vulnerabilidad crítica relacionada con la forma en que Lisp evalúa los argumentos:

Iteración 1: Falla de evaluación por Variable No Vinculada (Unbound Variable)

- **Análisis:** Al desarrollar el caso de prueba para forzar un error en la distribución porcentual (ingresando un parámetro alfanumérico en lugar de un número), el programa abortó su ejecución de manera fatal. En Lisp, el intérprete intenta evaluar los argumentos antes de aplicarlos a la función. Al no utilizar el operador de cotización en la letra A, el entorno asumió que se trataba del identificador de una variable viva en memoria. Al no encontrarla, lanzó la excepción Unbound variable A.

```

> (defun ejecutar-qa-completo ()
  (progn
    (format t "~%--- PRUEBAS REQ 6: DISTRIBUCION TEMPORAL ---~%" )
    (format t "Caso Normal (90s Rojo, 6s Amarillo, 120s Verde):~%" )
    (print (distribucion-porcentual 90 6 120))

    (format t "~%Caso Error (Letras en vez de numeros):~%" )
    (print (distribucion-porcentual A 6 120))
    'PRUEBAS-FINALIZADAS ))
)
EJECUTAR-QA-COMPLETO
> (ejecutar-qa-completo)

--- PRUEBAS REQ 6: DISTRIBUCION TEMPORAL ---
Caso Normal (90s Rojo, 6s Amarillo, 120s Verde):
((ROJO 41.66666666666667) (VERDE 55.55555555555556) (AMARILLO 2.7777777777777777))
Caso Error (Letras en vez de numeros):
Error: The variable A is unbound.

```

Iteración 2: Corrección mediante el operador de cotización (Versión Final)

- **Análisis:** Se solucionó el fallo modificando el entorno de pruebas mediante el agregado del operador de cotización simple ' (quote), transformando el argumento en 'A. De esta manera, se le indicó al evaluador que suspenda la evaluación lógica y trate al dato estrictamente como un símbolo literal inmutable. Esto permitió que el programa fluya y la función interceptara el error correctamente sin colapsar el entorno.

BITÁCORA DE BUGS Y JUSTIFICACIÓN

FASE 2

BITÁCORA - REQUERIMIENTO 1

Al comenzar el requerimiento 1, hice una versión muy simple de la función transicion.

El

principal problema fue que solamente contemplaba el caso en que el semáforo estaba en rojo y debía pasar a verde.

Cuando revisé el código me di cuenta de que cualquier otro caso devolvía NIL y eso no cumplía con lo pedido.

En la segunda versión agregué una validación para verificar tanto el estado actual como el color de destino.

Si

bien mejoró el funcionamiento, todavía seguía sin contemplar todos los escenarios posibles y continuaba devolviendo NIL en muchos casos.

En la tercera versión logré que la función devolviera una acción para rojo, amarillo y verde.

El inconveniente fue que repetía mucho código y cada nuevo color implicaba agregar más condiciones manualmente.

Aprendí que la repetición excesiva dificulta el mantenimiento del programa.

Finalmente, en la cuarta versión utilicé member

para validar los colores permitidos y concatenate junto con symbol-name para construir automáticamente el mensaje.

Esta solución me permitió reducir líneas de código, evitar repeticiones y hacer la función más flexible.

Lo que más me costó fue entender cómo convertir un símbolo en texto para generar la acción correspondiente.

Después de realizar varias pruebas comprendí mejor cómo trabajar con símbolos y cadenas de texto en LISP.

Considero que este requerimiento me ayudó a mejorar la organización del código y a pensar soluciones más generales.

BITÁCORA - REQUERIMIENTO 2

En

el requerimiento 2 tuve que desarrollar una función capaz de determinar automáticamente el color del semáforo según un tiempo Unix.

La primera versión fue muy básica y solamente distinguía entre rojo y verde.

Rápidamente observé que estaba ignorando el

color amarillo y que no existía un ciclo completo, por lo que el resultado no era correcto.

En la segunda versión agregué los tres colores utilizando `cond`. En ese momento comprendí que debía trabajar con la duración total del ciclo semafórico y no con el tiempo absoluto.

La tercera versión ya funcionaba correctamente, pero utilicé una variable global mediante `setq`. Después de analizar el código entendí que esa práctica no era recomendable porque podía generar efectos secundarios en otras partes del programa.

La versión final utiliza `let` para crear una variable local y `mod` para obtener la posición exacta dentro del ciclo de 216 segundos. Lo más difícil fue comprender cómo funciona la operación módulo para que el semáforo repita correctamente el ciclo sin importar qué valor de tiempo Unix se reciba.

Después de realizar distintos ejemplos y pruebas pude entender mejor el concepto de ciclos repetitivos.

Gracias

a este ejercicio aprendí a trabajar con temporización, operaciones modulares y variables locales, logrando un código más limpio y fácil de mantener.

BITÁCORA – Haskell P.2

Cuando empezamos con la parte de Haskell me encontré con varios problemas porque nunca había usado ese lenguaje. Al principio me costó bastante entender cómo funcionaba porque es diferente a los lenguajes que había visto antes.

Lo que más me confundió fue el tema de la Evaluación Perezosa (Lazy Evaluation). No entendía bien cómo un programa podía trabajar sin ejecutar todo de una sola vez. Después de buscar información y ver algunos videos en YouTube, fui entendiendo que Haskell va calculando solamente lo que necesita en cada momento.

También utilicé herramientas de inteligencia artificial para hacer consultas puntuales y pedir ejemplos más sencillos cuando algún concepto no me quedaba claro. Eso me ayudó a relacionar la teoría con el trabajo práctico que estábamos realizando.

Una vez que entendí mejor el concepto, pude relacionarlo con nuestra función `timer`. Me di cuenta de que, si hubiera muchísimos eventos de tráfico o

una secuencia muy grande de tiempos, Haskell no necesitaría procesarlos todos juntos, sino únicamente los que hicieran falta en ese momento.

Gracias a esta investigación aprendí conceptos nuevos sobre programación funcional y descubrí una forma diferente de resolver problemas. Aunque al principio me costó bastante entender el lenguaje, con la ayuda de videos, documentación y herramientas de apoyo pude comprender los conceptos necesarios para realizar esta parte del trabajo,

Además de investigar los conceptos teóricos de

Haskell, tuve que adaptar las funciones transicion y

timer que había desarrollado previamente en Common Lisp. Una de

las dificultades fue acostumbrarme a la sintaxis de

Haskell, ya que es bastante diferente. Comparando ambos lenguajes pude entender que, aunque la forma de escribir el código cambia,

la lógica del problema se mantiene. Esto me ayudó a comprender mejor tanto la solución desarrollada como las características propias de cada lenguaje.

ANÁLISIS FASE 3

1. Presentación del Lenguaje: Haskell

En la estructura fundamental de Haskell, la función principal siempre debe llamarse `main`.

```
1 main :: IO ()
2 main = putStrLn "¡Hola, Mundo!"
```

Características y Funcionalidades Básicas:

- **Funciones y Condicionales:** El lenguaje permite definir funciones matemáticas con declaración de tipos explícita (como sumar dos números) y manejar el flujo mediante sentencias `if-then-else`.

```
1 main :: IO ()
2 main = do
3     print (sumar 10 5)
4     putStrLn (esMayorDeEdad 20)
5
6 sumar :: Int -> Int -> Int
7 sumar x y = x + y
8
9 esMayorDeEdad :: Int -> String
10 esMayorDeEdad edad =
11     if edad >= 18
12     then "Es mayor de edad"
13     else "Es menor de edad"
```

Listas y Tuplas: Las listas en Haskell son colecciones homogéneas, lo que significa que todos los elementos en su interior deben pertenecer obligatoriamente al mismo tipo. Soportan operaciones estructurales comunes como extraer el primer elemento, obtener el resto de la colección o concatenar. Por el contrario, las tuplas poseen un tamaño fijo pero son heterogéneas, permitiendo mezclar distintos tipos de datos (como representar el nombre en

texto y la edad en números de un usuario).

```
1 main :: IO ()
2 main = do
3     --defino lista
4     let numeros = [1, 2, 3, 4, 5] :: [Int]
5
6     --operaciones mas comunes
7     primero = head numeros -- Devuelve 1
8     resto = tail numeros -- Devuelve [2, 3, 4, 5]
9     agregar = 0 : numeros -- Agrega el 0 al principio: [0, 1, 2, 3, 4, 5]
10    concatenar = numeros ++ [6] -- Une listas: [1, 2, 3, 4, 5, 6]
11
12 --IMPRIMO
13    print numeros --me devuelve la lista
14    print primero
15    print resto
16    print agregar
17    print concatenar
```

```
1 main :: IO ()
2 main = do
3     let usuario :: (String, Int)
4         usuario = ("Juan", 25)
5
6     print usuario
```

Pattern Matching y Guardas: Para evitar el anidamiento excesivo de sentencias `if`, Haskell implementa *Pattern Matching*, una técnica que busca la coincidencia directa de los argumentos. Como complemento, utiliza *Guardas*, una estructura similar a un `switch` pero orientada a evaluar condiciones verdaderas o falsas de manera secuencial.

```
1
2 -- Definir el factorial usando Pattern Matching y Recursión
3 factorial :: Int -> Int
4 factorial 0 = 1
5 factorial n = n * factorial (n - 1)
6
7 -- Pattern matching con listas
8 estaVacia :: Show a => [a] -> String
9 estaVacia [] = "La lista no tiene elementos"
10 estaVacia (x:xs) = "El primer elemento es " ++ show x
11
12 main :: IO ()
13 main = do
14     print (factorial 5)
15     putStrLn (estaVacia ([] :: [Int]))
16     putStrLn (estaVacia [10, 20, 30])
```

Funciones de Orden Superior: El ecosistema funcional destaca por el uso intensivo de abstracciones como `map` (que aplica una transformación a cada

elemento de una colección) y `filter` (que filtra y extrae exclusivamente los elementos que cumplen una condición dada).

```
1 listaOriginal :: [Int]
2 listaOriginal = [1, 2, 3, 4, 5]
3
4 -- map: Aplica una función a cada elemento
5 duplicados :: [Int]
6 duplicados = map (\x -> x * 2) listaOriginal
7 -- Resultado: [2, 4, 6, 8, 10]
8
9 -- filter: Filtra los elementos que cumplen una condición
10 pares :: [Int]
11 pares = filter even listaOriginal
12 -- Resultado: [2, 4]
13
14 main :: IO ()
15 main = do
16     print listaOriginal
17     print duplicados
18     print pares
```

Uso en la Industria y Áreas de Aplicación:

Sector Financiero y Fintech: Las finanzas representan el hogar corporativo por excelencia para Haskell. Al manejar algoritmos matemáticos complejos y transacciones masivas, las instituciones necesitan un compilador estricto que detecte errores antes del pase a producción. Un caso de éxito es Standard Chartered, que mantiene millones de líneas de código en este lenguaje para sus sistemas de gestión de riesgos y valoración de activos.



Tecnología Blockchain y Criptomonedas: La inmutabilidad inherente a la programación funcional encaja de manera perfecta con la filosofía de los registros distribuidos. Por ejemplo, toda la red de Cardano (ADA) y su plataforma de contratos inteligentes (Plutus) están desarrolladas íntegramente sobre Haskell.



Big Tech (Infraestructura Crítica): Aunque las grandes tecnológicas no suelen usar Haskell para el código de cara al usuario final, sí lo implementan en sus herramientas internas más críticas, donde el procesamiento de datos a gran escala es vital. Meta (Facebook) lo utiliza en *Sigma* (una plataforma anti-abuso que detecta spam y malware procesando millones de peticiones) y en *Glean* (herramienta de análisis de código fuente).



Ciberseguridad y Análisis de Código: Gracias a sus potentes librerías de parsing (como Parsec), Haskell facilita enormemente la creación de herramientas de seguridad y análisis estático. *ShellCheck*, la reconocida herramienta para detectar errores en scripts de Bash, está escrita enteramente en este lenguaje.

Academia e Investigación: Haskell funciona como el laboratorio principal donde se prueban las teorías de tipos más avanzadas antes de su adopción general en la industria, habiendo inspirado conceptos como las funciones lambda incorporadas posteriormente en Python y Java.

RESPUESTAS A PREGUNTAS TEÓRICAS FASE 3

Respuesta a Pregunta 1 (Tipado Dinámico vs ADT): La diferencia fundamental radica en que Common Lisp utiliza un tipado mayormente dinámico (verificando los tipos de datos sobre la marcha durante la ejecución), mientras que Haskell posee un tipado estático Estricto. Esto significa que el compilador analiza íntegramente el código y detecta los errores de tipo antes de que el programa pueda siquiera ejecutarse.

Para modelar nuestro sistema, los colores del semáforo fueron definidos en Haskell mediante un *Algebraic Data Type* (ADT) nombrado `Color`, conteniendo los valores fijos de Rojo, Amarillo y Verde. Un ADT es un tipo de dato definido por el programador que permite representar un conjunto específico y estrictamente limitado de valores válidos. En este caso, el tipo `Color` encapsula los únicos estados posibles del semáforo. Gracias a este sistema, funciones como la transición de estados solo podrán recibir valores pertenecientes al tipo `Color`. Si un programador intentara ingresar un valor inválido (por ejemplo, "Rosado"), el compilador detectaría la anomalía inmediatamente y abortaría la compilación. En conclusión, el uso de un ADT respaldado por el tipado estático garantiza que el semáforo opere exclusivamente en estados válidos, bloqueando errores de forma preventiva y elevando la confiabilidad del programa.

Respuesta a Pregunta 2 (Evaluación Perezosa / Lazy Evaluation): La Evaluación Perezosa (*Lazy Evaluation*) es una característica de Haskell que permite que los valores se calculen única y exclusivamente cuando son requeridos por el sistema. Este enfoque resulta crítico al trabajar con flujos de datos inmensos o infinitos (como eventos de tráfico), ya que el sistema se exime de procesar toda la carga de una sola vez.

En nuestro proyecto, la función `timer` (desarrollada en el Requerimiento 2) opera como una función pura, ya que su resultado depende de manera exclusiva del tiempo recibido como parámetro. Esta naturaleza pura es ideal para trabajar con Lazy Evaluation, dado que el lenguaje puede diferir los cálculos sin riesgo de disparar efectos secundarios impredecibles. Al recibir un tiempo Unix, `timer` determina el color utilizando la duración total del ciclo. Si lleváramos esta lógica a Haskell bajo un escenario de monitoreo continuo, podríamos enfrentarnos a una secuencia infinita de *timestamps* representando el avance del tiempo. En lugar de colapsar intentando procesar el futuro, Haskell calcularía el resultado de la función `timer` solamente para los instantes de tiempo que sean explícitamente consultados, evitando ejecutar rutinas innecesarias sobre el resto de la secuencia infinita. La ventaja absoluta de este modelo es el drástico ahorro de memoria y recursos de procesamiento.

Conclusión GRUPAL:

Como equipo, consideramos que el desarrollo de este trabajo práctico representó una experiencia sumamente exigente. Nos impulsó a coordinar tareas entre compañeros que no nos conocíamos previamente y a incorporar rápidamente el uso de herramientas profesionales como Git y GitHub. A nivel técnico, dominar un nuevo lenguaje de programación como Haskell en un período tan corto nos exigió investigar y afianzar de manera autodidacta tanto su particular sintaxis como el enfoque del paradigma funcional.

Más allá de las competencias técnicas (*hard skills*), el proyecto nos presentó el gran desafío de perfeccionar nuestras habilidades blandas. Aprendimos a fomentar la comunicación constante, ejercer el respeto ante los distintos puntos de vista, mantener la empatía con la situación individual de cada integrante, desarrollar una fuerte adaptabilidad frente a los errores del código, y por, sobre todo, a gestionar eficientemente nuestros tiempos para resolver los conflictos que surgieron en el camino hacia la entrega final.

Bibliografía:

Fuentes Principales (Material de la Cátedra)

1. Monzón, R. (2026). Paradigmas y Lenguajes - Introducción. Facultad de Ciencias Exactas y Naturales y Agrimensura - UNNE.

Contenido: Conceptos fundamentales de paradigmas de programación y clasificación de lenguajes.

2. Monzón, R. (2026). *Paradigmas y Lenguajes - Programación Funcional Parte 1. Facultad de Ciencias Exactas y Naturales y Agrimensura - UNNE.

Contenido: Introducción a LISP, tipos de datos (átomos, listas, conses), funciones aritméticas básicas (+, -, *, /), funciones de truncamiento (truncate, round) y predicados numéricos (numberp, integerp, zerop, evenp, oddp).

3. Monzón, R. (2026). *Paradigmas y Lenguajes - Programación Funcional Parte 2. Facultad de Ciencias Exactas y Naturales y Agrimensura - UNNE.

Contenido: Funciones para operar sobre listas (car, cdr, cons, append), funciones de asignación (setq), y predicados de igualdad (eq, eql, equal).

4. Monzón, R. (2026). *Paradigmas y Lenguajes - Programación Funcional Parte 3. Facultad de Ciencias Exactas y Naturales y Agrimensura - UNNE.

Contenido: Estructuras condicionales (if, cond, when, case), estructuras iterativas (do, dotimes, dolist), entrada/salida simple (format, print, read) y funciones de orden superior (mapcar).